

1 Einleitung und Beschreibung des Projekts

1.1 Einleitung

In „Software Engineering II“ haben wir auf den bisher erlernten Grundlagen aufgebaut und unser Verständnis für zentrale Aspekte der Softwareentwicklung vertieft – von der Entwicklungsumgebung über Nebenläufigkeit und Datenbankanbindung bis hin zur systematischen Problemfindung. Ein besonderer Fokus lag auf der Vertiefung objektorientierter Konzepte: Durch die intensive Arbeit mit dem Jakarta-Servlet-API, Vererbung und Design Patterns haben wir gelernt, wiederverwendbaren und gut strukturierten Code zu schreiben, der uns zugleich hilft, Probleme systematisch zu analysieren und im Team zu lösen.

Im Folgenden präsentieren wir die Ergebnisse der Semesteraufgabe: eine Bibliotheksanwendung, die es Mitarbeitern ermöglicht, sich anzumelden und zwischen den Anwendungsfällen Ausleihe und Rückgabe zu wählen. Über die geforderten Kernfunktionen hinaus haben wir eine eigenständige Registrierung für Mitglieder ergänzt, die eng mit dem bestehenden Login-System verzahnt ist, sodass neu registrierte Mitglieder unmittelbar im System anmelden und ihre Ausleihen einsehen können.

1.2 Beschreibung des Projekts

Den Kern der Anwendung bilden die beiden geforderten Anwendungsfälle Ausleihe und Rückgabe. Mitarbeiter geben dabei zunächst den Ausweiscode eines Mitglieds ein, gefolgt von einer beliebigen Anzahl an Buchexemplar-Codes, die per Tastatur eingegeben werden, um einen Barcode-Scanner zu simulieren. Erst nach ausdrücklicher Bestätigung der vollständigen Erfassung wird die Ausleihe beziehungsweise Rückgabe tatsächlich in der Datenbank festgehalten. Bestätigungsdokumente werden dabei nicht unmittelbar geschrieben, sondern als eigenständiges Objekt gekapselt und in eine Redis-Warteschlange eingereiht; ein separater, nebenläufig laufender Prozess entnimmt diese Aufträge und schreibt die entsprechenden Quittungen als Textdateien.

Über die Kernfunktionen hinaus haben wir die Anwendung um mehrere Module erweitert, die reale Bibliotheksprozesse abbilden: Bei der Inventur erfassen Mitarbeiter Buchexemplare je Standort, wobei das System automatisch über- sowie unterzählige Bestände ermittelt; ein Mitglieder-Dashboard zeigt die eigene Ausleihhistorie, während ein administrativer Bereich Bestand und Anschaffungen verwaltet. Zur Sicherung der Codequalität haben wir zudem einen Git-Hook eingerichtet, der falsch formatierten Java-Code bereits vor dem Push zurückweist, und

den vollständigen Ablauf von Anmeldung über eine Aktion bis zur Abmeldung als Grundlage für einen nebenläufigen Lasttest genutzt, dessen Ergebnisse wir visualisiert und ausgewertet haben. Für die Entwicklung der Benutzeroberfläche wurden HTML, CSS und JavaScript verwendet [1]. [2].

2 Anforderungsanalyse

2.1 Stakeholder

Im Rahmen der Bibliotheksanwendung lassen sich folgende Stakeholder identifizieren:
Mitglieder – Nutzer der Bibliothek, die Bücher ausleihen, zurückgeben und ihren Kontostatus einsehen möchten.
Mitarbeiterinnen und Mitarbeiter – Bibliothekspersonal, das Ausleihen und Rückgaben durchführt, Mitglieder verwaltet und bei Bedarf sperren oder freigeben kann.
Administratorinnen und Administratoren – Mitarbeitende mit erweiterten Rechten, die Statistiken einsehen, Bücher verwalten und Auswertungen durchführen können.

2.2 Personas

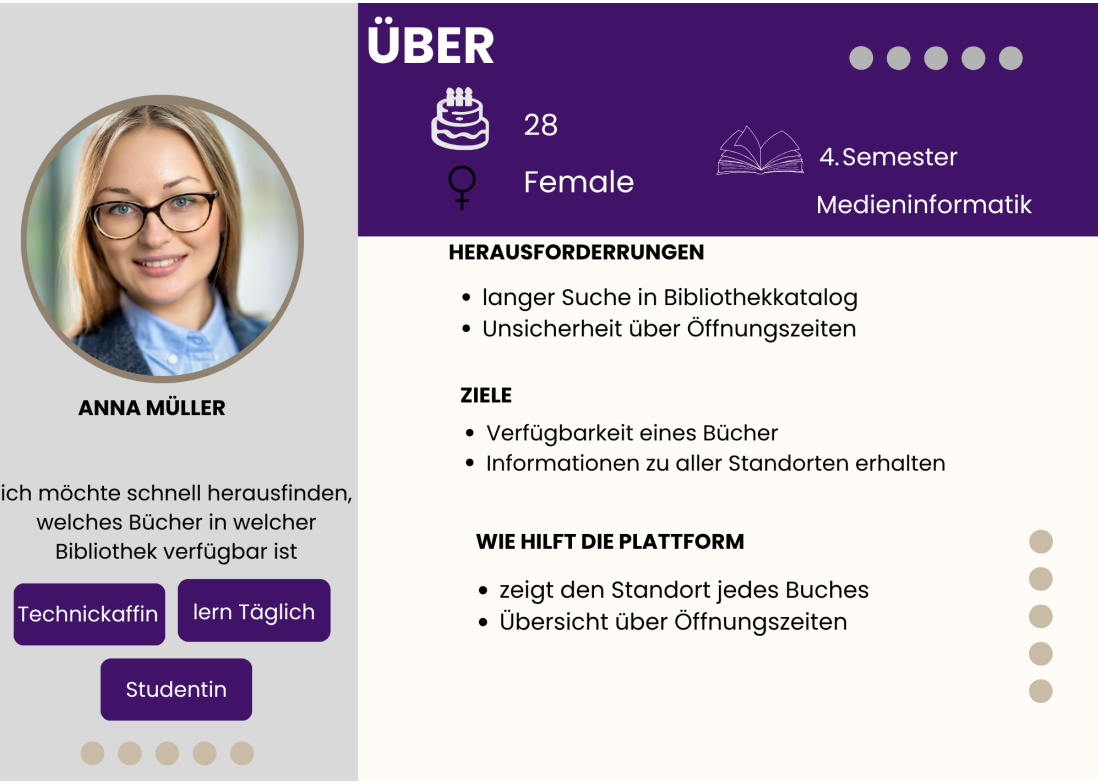


Abbildung 1: Persona 1

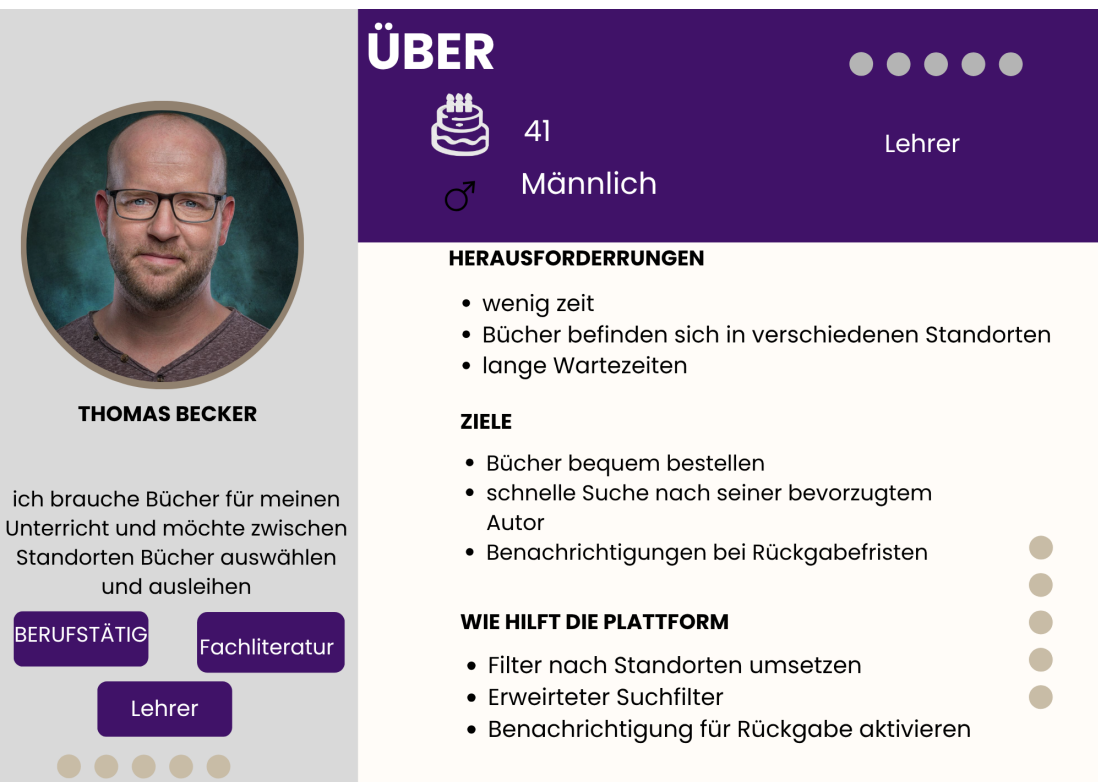


Abbildung 2: Persona 2

2.3 Anforderungsanalyse

2.3.1 Funktionale Anforderungen

Mitglieder: Mitglieder können sich registrieren und einloggen. Auch ohne Anmeldung ist es möglich, Bücher zu suchen und deren Verfügbarkeit nach Standort und Regal einzusehen. Nach dem

Login haben Mitglieder Zugriff auf ihre aktiven Ausleihen mit Fälligkeitsdatum und anfallenden Mahngebühren sowie auf ihre Ausleihhistorie. Gesperrte Mitglieder erhalten eine Anzeige mit dem jeweiligen Sperrgrund.

Mitarbeiterinnen und Mitarbeiter: Mitarbeitende authentifizieren sich mit E-Mail, Passwort und Mitarbeiter-ID. Sie erfassen Ausleihen, indem sie die Ausweisnummer des Mitglieds und mehrere Barcodes nacheinander einscannen und die Ausleihe anschließend bestätigen. Bei der Rückgabe können mehrere Bücher gleichzeitig bearbeitet werden, wobei der Zustand jedes Buches erfasst wird. Mahngebühren werden automatisch mit 0,10 Euro pro Tag nach Ablauf der Leihfrist berechnet. Darüber hinaus können Mitarbeitende Mitglieder suchen, ihre Ausleihhistorie einsehen sowie Mitglieder manuell mit Angabe eines Grundes sperren oder freigeben.

Administratorinnen und Administratoren: Admins haben Zugriff auf eine Übersicht mit Kennzahlen wie Bestand, aktive Ausleihen, überfällige Ausleihen und gesperrte Mitglieder. Sie können Bücher suchen, hinzufügen – mit Angabe von Autor, Standort, Regalnummer und Exemplaranzahl – sowie aus dem Bestand entfernen. Zusätzlich stehen Autorenstatistiken, gefilterte Mitgliederübersichten und Auswertungen nach Zeitraum zur Verfügung.

2.3.2 Nicht-funktionale Anforderungen

Sicherheit: Authentifizierung erfolgt sitzungsbasiert; Session Fixation wird verhindert. Der Zugriff auf geschützte Seiten wird serverseitig durch einen Servlet-Filter (MyFilter) geregelt. Nicht autorisierte Zugriffsversuche werden auf die Login-Seite umgeleitet.

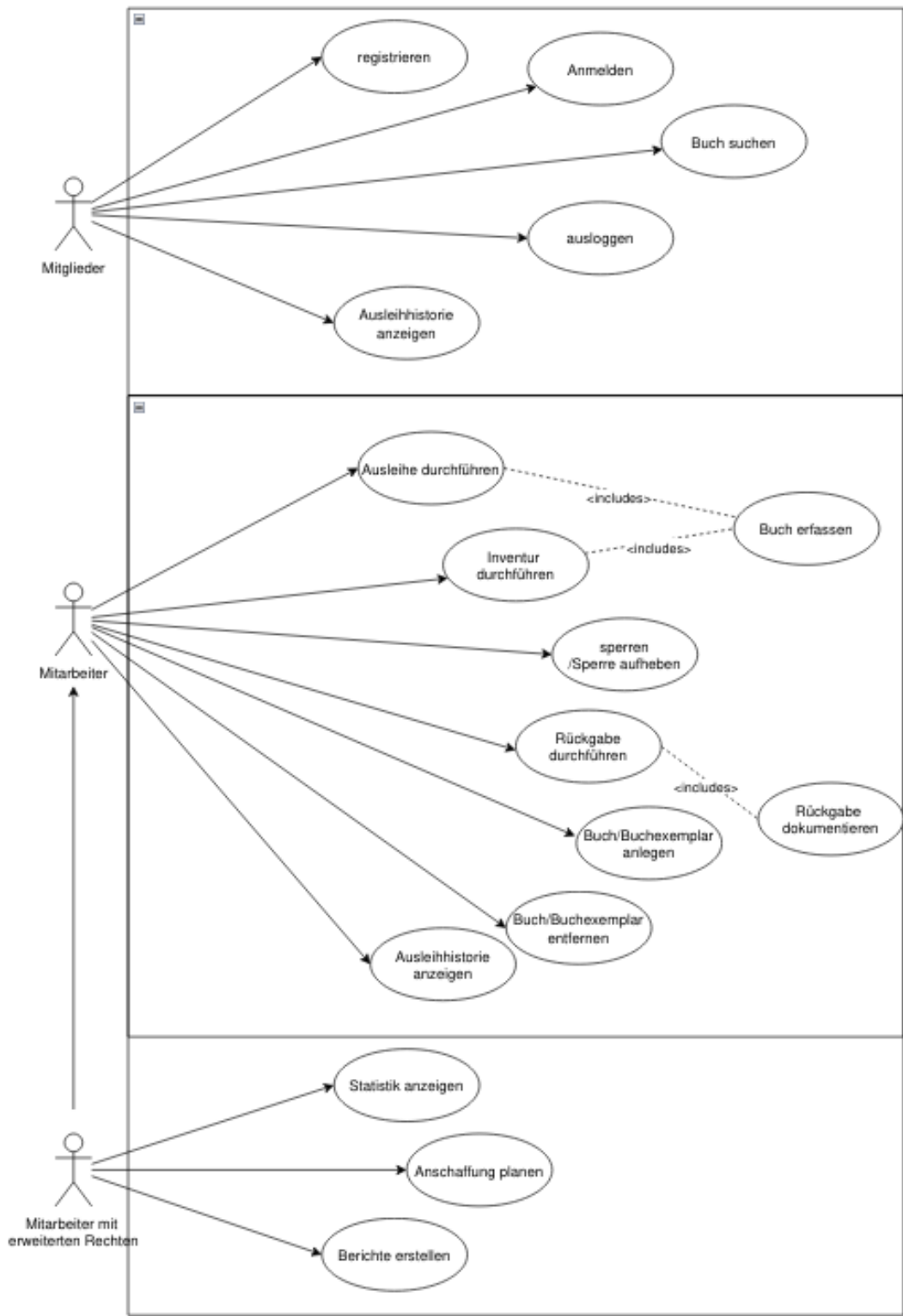


Abbildung 3: Use Case

3 Design pattern

Design Patterns bezeichnen Linien von Code, die immer wieder genutzt werden, um gewisse Lösungen für auftretende Probleme oder Herausforderungen in der Softwareentwicklung zu gewährleisten. Durch den Einsatz von Design Patterns wird der Code übersichtlicher, besser wartbar und leichter erweiterbar. In unserer Anwendung haben wir das Pattern **Singleton** umgesetzt, das zur Erzeugungsmuster-Kategorie gehört.[3] "Das Singleton Pattern stellt sicher, dass von einer Klasse nur eine einzige Instanz existiert und bietet einen globalen Zugriffspunkt auf diese Instanz" beschreibt (Huber, 2026).

```
1 public class DesignServlet extends
2     ↳ HttpServlet{
3     private DataSource ds;
4
5     @Override
6     public void init() throws
7         ↳ ServletException {
8         try {
9             Context ctx = new InitialContext();
10            ds = (DataSource) ctx.lookup("java:
11            ↳ /comp/env/jdbc/mariadb");
12        } catch (Exception e) {
13            throw new ServletException(e);
14        }
15    }
16
17    @Override
18    public void doGet(HttpServletRequest
19        ↳ req, HttpServletResponse res)
20        throws ServletException, IOException {
21        connection con = ds.getConnection();
22    }
```

Datasource wird in unserer Webanwendung zentral verwendet, auf den alle Servlet zugreifen können. Somit werden Ressourcen gespart, da bei jeder Anfrage nicht mehr direkt neue Datenbankverbindungen aufgemacht werden müssen.

4 Analyse Muster von Heide Balzert

Heide Balzert (* 31 Januar 1955 in Essen) ist eine deutsche Informatikerin und Professorin im Bereich **Softwaretechnik und Systemanalyse**. In eines ihrer Bücher "Lehrbuch der Objektmodellierung" untersucht sie verschiedene AnalyseMuster und Beispielanwendungen, uns interessiert prinzipiell der Muster nach dem **Exemplar-typ**. Balzert beschreibt den Exemplartyp als

- Einfache Assoziation
- erstellte Objektverbindungen nicht verändert werden und werden nur gelöscht, wenn das betreffende Exemplar gelöscht wird
- Name der neuen Klasse enthält oft Begriffe wie Typ, Gruppe und Beschreibung

Dieses Muster untersucht das Problem, dass mehrere Objekten zum gleichen Typ gehören und gemeinsame Eigenschaften besitzen. Laut der *UML-Diagramm in dem Buch* werden verschiedene Buchexemplare dargestellt, die gleiche Buchbeschreibungen haben. Balzert sprach vom Redundante Speicherung von Attributwerten, da die in der Klasse gespeicherten Informationen immer wieder wiederholt werden. Die Abbildung eine

zweite Klasse, die die Eigenschaften *Titel, Autor und Verlag* als Attributen enthält während Buchexemplar [4] nur Daten für diese Exemplare besitzt, bildet eine Lösung für das oben genannte Problem. Durch diese Lösung können Redundanzen vermieden werden und somit ist das Modell übersichtlicher.

5 Architekturbild & Buildprozess

Die Anfragen werden vom Browser über HTTPS an den **HAProxy**-Loadbalancer gesendet. Dieser verteilt sie an den Docker-Container, in dem **Tomcat** als Application-Server läuft. Tomcat verarbeitet die Anfragen und greift dabei auf **Redis** als Warteschlange für Hintergrundaufgaben sowie über **JDBC** auf die **MariaDB** zur persistenten Datenspeicherung zu. Links unten ist die Entwicklungsumgebung mit der Verzeichnisstruktur (*bin/, build/, src/, app/, target/ und local/*) dargestellt. Von dort aus wird die Anwendung erstellt (Build) und über den **Tomcat Manager** im Docker-Container bereitgestellt (Deploy).

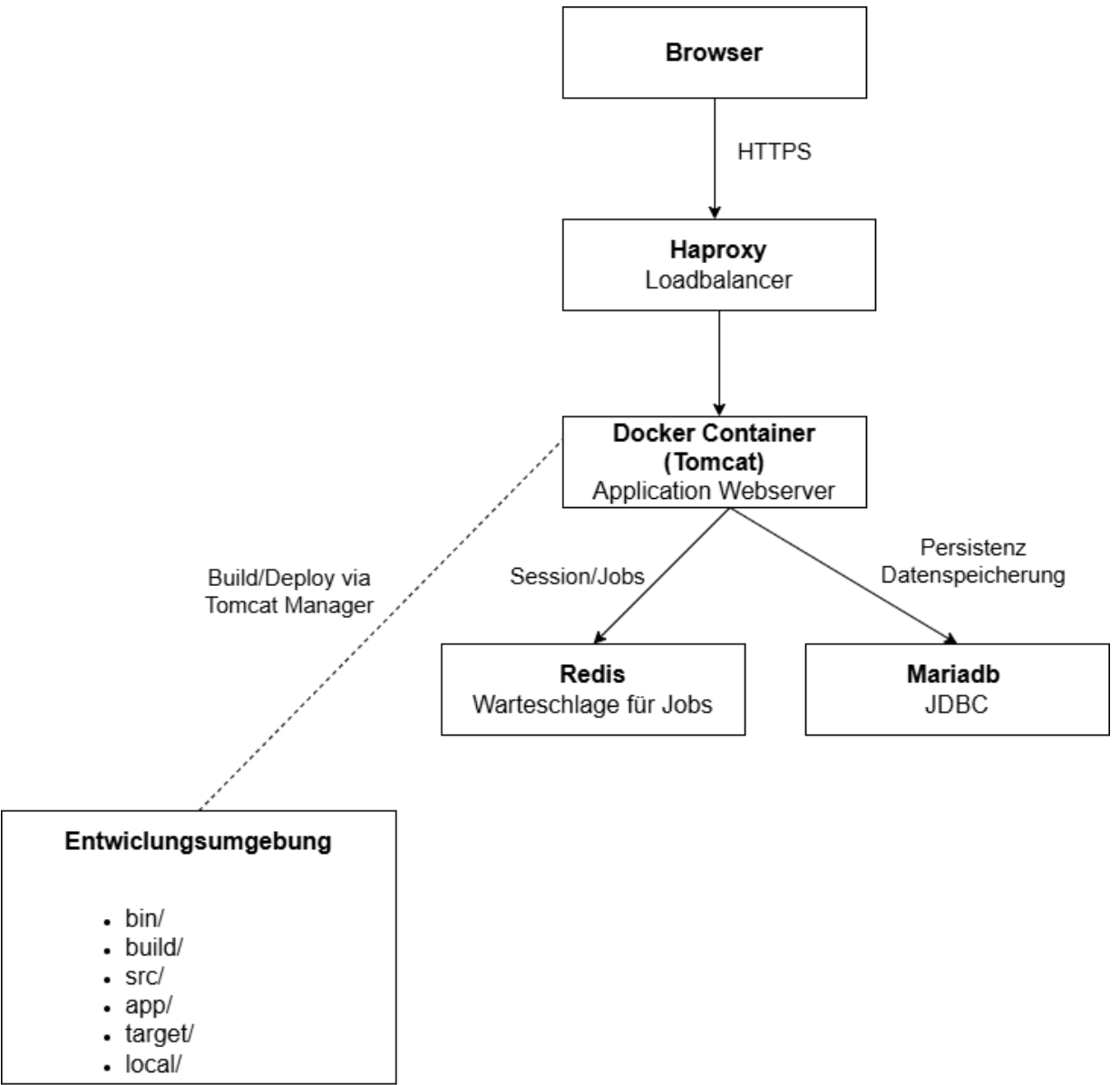


Abbildung 4: Architekturbild

Buildprozess:

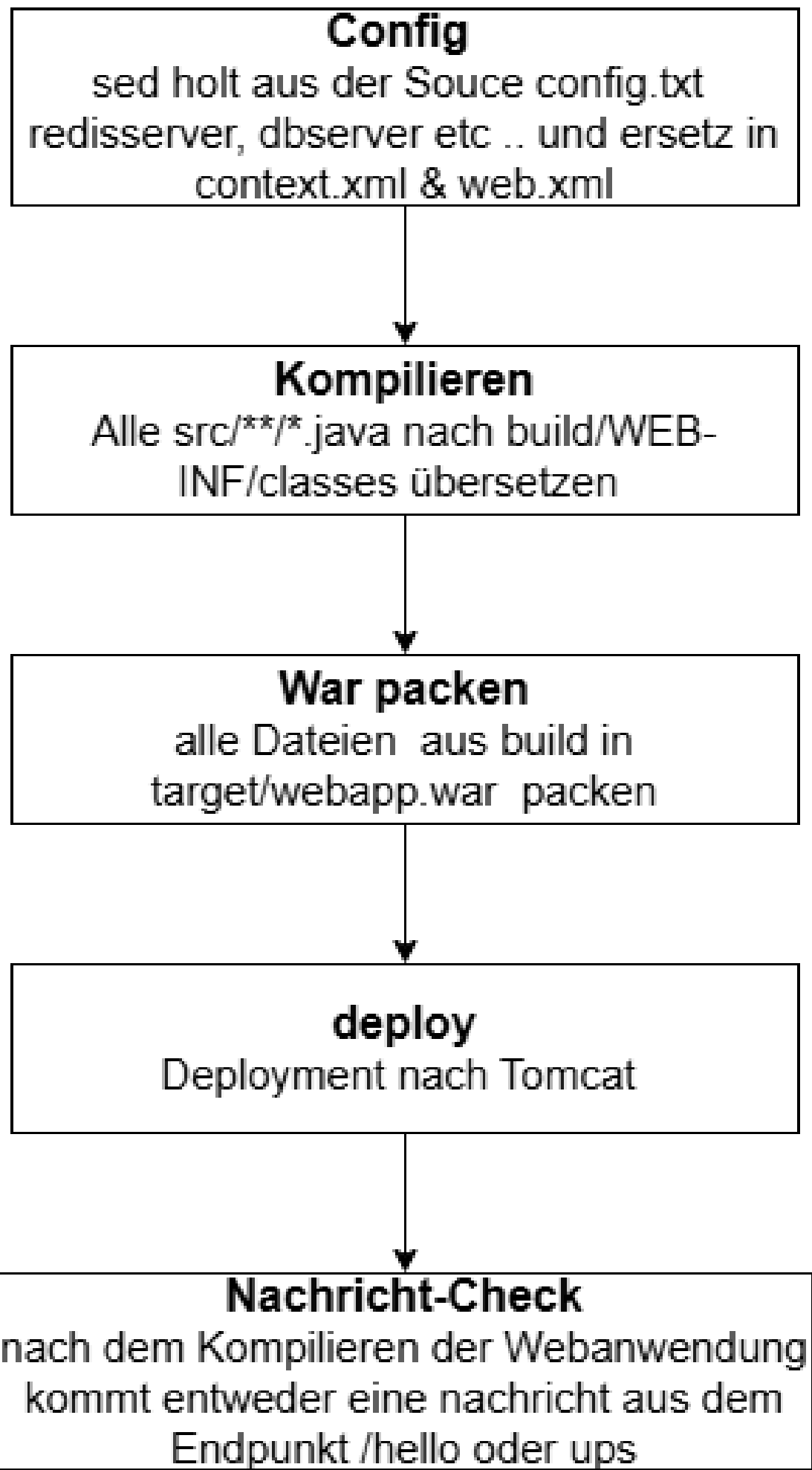


Abbildung 5: build-Prozess

6 Aktivitätsdiagramm

Wir haben ein Aktivitätsdiagramm erstellt, dass den kompletten Ablauf vom Login über die Ausleihe eines Mitarbeiters zeigt.

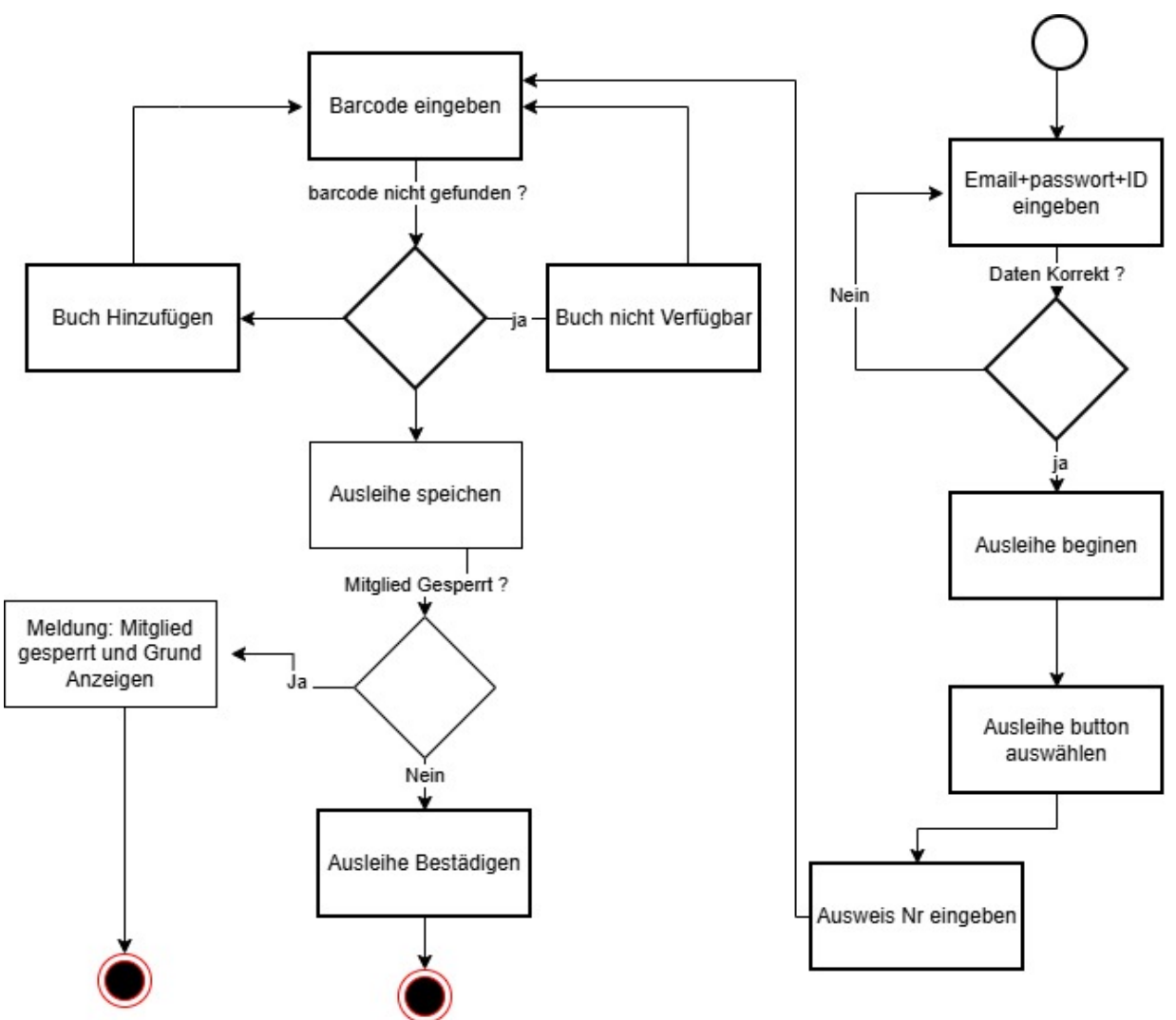


Abbildung 6: Aktivitätsdiagramm: Login und Ausleihe

7 Umsetzung & Kurze Beschreibung

7.1 Ausleihe

Die Ausleihe ist das Herzstück unserer Bibliotheksanwendung. Der Mitarbeiter gibt die Ausweisnummer des Mitglieds ein und scannt anschließend beliebig viele Barcodes der auszuleihenden Bücher. Erst ein Klick auf „Alle ausleihen“ speichert alle Bücher gemeinsam in der Datenbank. Statt die Bestätigung sofort auszudrucken, wird ein Druckauftrag mit allen Angaben – Mitarbeitername, Ausweisnummer, ausgeliehene Bücher sowie Ausleih- und Rückgabedatum – als eigenständiges Objekt gekapselt und per LPUSH in eine Redis-Warteschlange eingereiht.

```
1 while (laeuft) {
2     try {
3         RedisClient redisClient =
4             ↳ JedisAdapter.getClient();
5         List<String> antwort =
6             ↳ redisClient.brpop(WARTE_SEKUNDEN,
7             ↳ QUEUE_KEY);
8         if (antwort == null) {
9             continue;
10        }
11        String auftragAlsText =
12            ↳ antwort.get(1);
13        schreibeQuittung(auftragAlsText);
14    } catch (Exception fehler) {
15        System.err.println("Fehler
16        ↳ beim Verarbeiten: " +
17        ↳ fehler.getMessage());
18    }
19    System.out.println("DruckWorker
20    ↳ beendet.");
21}
```

Ein separater DruckWorker-Thread wartet kontinuierlich mit einem blockierenden BRPOP auf neue Aufträge in der Warteschlange, statt die Queue in einer Schleife ständig abzufragen. Sobald ein Auftrag eintrifft, entnimmt der Worker ihn und schreibt die entsprechende Quittung als

Textdatei. So bleibt die Ausleihe für den Mitarbeiter sofort abgeschlossen, während das eigentliche „Drucken“ nebenläufig im Hintergrund erfolgt.

7.2 Rückgabe

Bei der Rückgabe gibt der Mitarbeiter die Ausweisnummer des Mitglieds sowie beliebig viele Barcodes auf einmal ein, sodass mehrere Bücher in einem Vorgang zurückgenommen werden können. Für jedes Buch prüft die Anwendung automatisch, ob die Rückgabe überfällig ist, und berechnet gegebenenfalls eine Mahngebühr von 10 Cent pro Tag. Wird ein Buch als beschädigt oder verloren gemeldet, sperrt das System das betroffene Mitglied automatisch. Analog zur Ausleihe wird auch hier kein Beleg direkt geschrieben, sondern ein Druckauftrag in dieselbe Redis-Warteschlange eingereiht und vom DruckWorker verarbeitet.

```
1 long tageUeberfaellig =
  ↳ ChronoUnit.DAYS.between(faellig, heute);
2 double gebuehr = tageUeberfaellig > 0 ?
  ↳ tageUeberfaellig * 0.10 : 0.0;
3 if (!zustand.equals("GUT")) {
4   PreparedStatement psSperr =
     ↳ con.prepareStatement(
5     "UPDATE mitglied SET gesperrt=TRUE,
     ↳ sperr_grund=? WHERE id=?");
6   psSperr.setString(1, "Buch " +
     ↳ zustand.toLowerCase() + "
     ↳ zurueckgegeben");
7   psSperr.setLong(2, mitglied_id);
8   psSperr.executeUpdate();
9 }
```

Beide Prüfungen laufen automatisch im Hintergrund ab: Der Mitarbeiter muss weder die Gebühr selbst berechnen noch manuell entscheiden, ob ein Mitglied gesperrt werden soll. Die Sperrung erfolgt dabei unabhängig von der Gebühr allein aufgrund des gemeldeten Buchzustands.

7.3 Registrierung und Login

Das Klassendiagramm zeigt die drei zentralen Servlets für Registrierung und Anmeldung: RegisterServlet, MitgliedLoginServlet und MitarbeiterLoginServlet. Alle drei erben von HttpServlet und greifen über eine gemeinsame DataSource auf dieselbe MariaDB-Datenbank zu. Das RegisterServlet überschreibt doPost(), da die Registrierungsdaten per POST übertragen werden; die beiden Login-Servlets überschreiben stattdessen doGet(), um die eingegebenen Anmeldedaten zu verarbeiten.

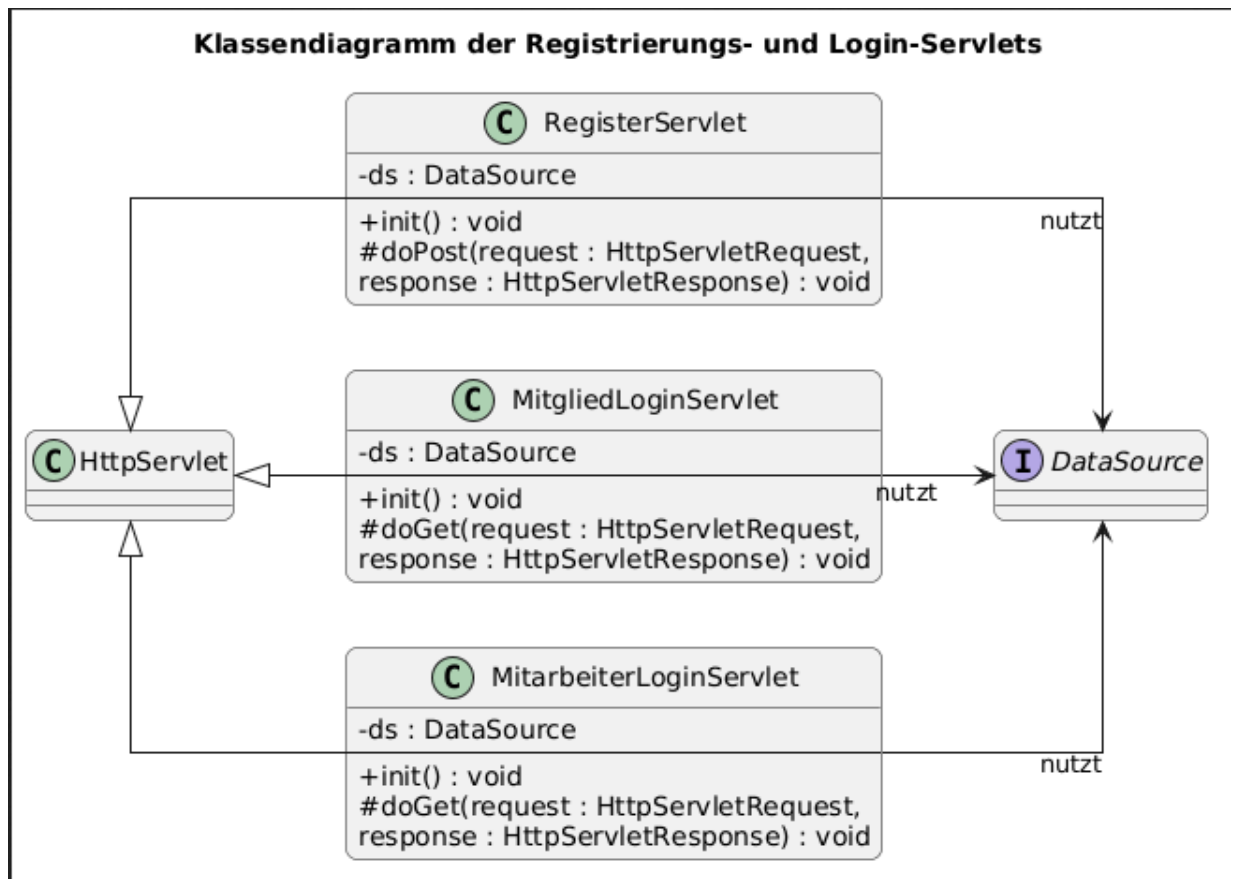


Abbildung 7: Klassendiagramm: RegisterServlet, MitgliedLoginServlet und MitarbeiterLoginServlet

Das RegisterServlet prüft zunächst, ob alle Eingaben vollständig sind und die beiden E-Mail-Felder übereinstimmen. Da zum Zeitpunkt des Einfügens die künftige Datenbank-ID noch nicht bekannt ist, wird das Mitglied zunächst mit einem Platzhalter für die Ausweisnummer angelegt und diese anschließend anhand der automatisch generierten ID nachgetragen. Die gesamte Operation läuft innerhalb einer Transaktion: Tritt ein Fehler auf, wird sie vollständig zurückgesetzt, sodass kein Mitglied mit unvollständigen Daten in der Datenbank verbleibt.

```
1 insertStmt.setString(5, "TEMP");
2 insertStmt.executeUpdate();
3 long generatedId;
4 try (ResultSet keys =
  ↳ insertStmt.getGeneratedKeys()) {
5   keys.next();
6   generatedId = keys.getLong(1);
7 }
8 String ausweisnummer =
  ↳ String.format("M-%06d", generatedId);
9 try (PreparedStatement updateStmt =
  ↳ connection.prepareStatement(
10    "UPDATE mitglied SET ausweisnummer =
     ↳ ? WHERE id = ?")) {
11   updateStmt.setString(1, ausweisnummer);
12   updateStmt.setLong(2, generatedId);
13   updateStmt.executeUpdate();
14 }
15 connection.commit();
```

Das MitgliedLoginServlet vergleicht die eingegebene E-Mail-Adresse und das Passwort mit den gespeicherten Daten. Bei erfolgreicher Anmeldung wird eine Session mit Mitglieds-ID, Ausweisnummer und Rolle angelegt, bevor das Mitglied auf sein Dashboard weitergeleitet wird. Das MitarbeiterLoginServlet funktioniert nach demselben Prinzip, leitet nach erfolgreicher Anmeldung jedoch abhängig von der Rolle entweder auf die Administrator- oder auf die Mitarbeiterseite weiter.

8 Lasttest

Der Lasttest simuliert eine realistische Nutzungslast auf dem System. Jeder parallele Prozess führt dabei einen vollständigen Ablauf durch: Login als Mitarbeiter, Rückgabe eines Buches und anschließender Logout. Jeder Mitarbeiter arbeitet mit einem eigenen Mitglied und Barcode, um Konflikte zu vermeiden. Die Gesamtdauer wird in Millisekunden gemessen und zusammen mit der Anzahl erfolgreicher und fehlgeschlagener Anfragen in einer CSV-Datei gespeichert.

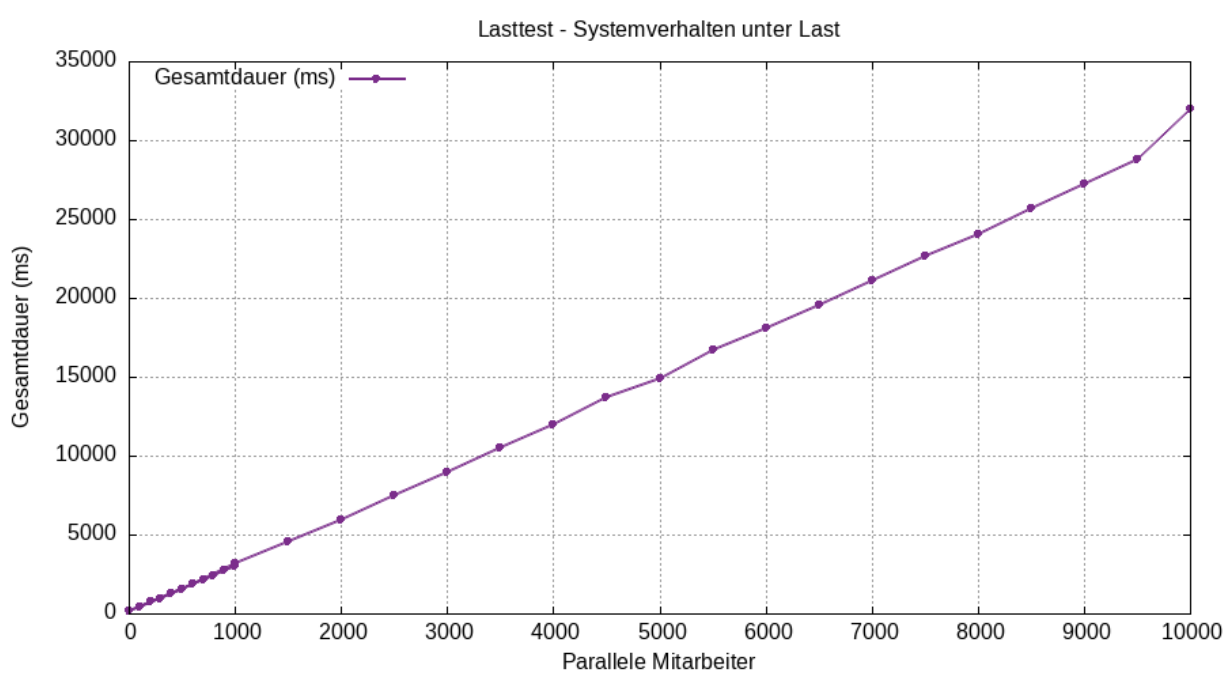


Abbildung 8: Lasttest

Eine nahezu lineare Kurve zeigt, dass die Infrastruktur (Tomcat, DB-Verbindungspool) auch unter steigender Last stabil und ohne Sättigung skaliert.

9 GitHook

Ein pre-push-Hook prüft vor jedem git push automatisch alle Java-Dateien im Repository mit Checkstyle. Enthält eine Datei Formatierungsfehler, wird der Push abgebrochen.

```
1 for file in $(find "$ROOT/brotundbutter-ses_
  ↳ sions-redis-mariadb/src" -name
  ↳ "*.java"); do
2   result=$(java -jar "$CHECKSTYLE" -c
     ↳ "$CONFIG" "$file" 2>&1)
3   if echo "$result" | grep -q "ERROR"; then
4     echo "Formatierungsfehler in: $file"
5     FEHLER=1
6   fi
7 done
8
9 if [ $FEHLER -eq 1 ]; then
10  echo ""
11  echo "Push abgebrochen. Bitte
     ↳ Formatierungsfehler beheben."
12  exit 1
13 fi
```

10 Zusammenfassung

Durch dieses Projekt konnten wir wesentliche Konzepte des Software Engineerings praktisch anwenden. Mithilfe von Personas, Anforderungsanalysen, dem Use-Case und dem Aktivitätsdiagramm wurden die Anforderungen systematisch erfasst und in eine strukturierte Webanwendung überführt. Die Kombination aus Servlets, MariaDB und Redis ermöglichte eine effiziente Umsetzung der Geschäftslogik sowie eine Druckwarteschlange nach dem Producer-Consumer-Muster. Zur Qualitätssicherung integrierten wir einen Git-Hook, der vor jedem Push die Java-Formatierung prüft, sowie Lasttests, die das Systemverhalten unter paralleler Belastung analysieren. Das Projekt hat gezeigt, dass Softwarequalität durch das Zusammenspiel von Implementierung, Sicherheit, Versionsverwaltung und Performanceanalyse entsteht.

Referenzen

[1] „JavaScript , Json, css, Html.“ en. Adresse: https://www.w3schools.com/js/js_json.asp.

[2] „Java.*.“ en. Adresse: <https://docs.oracle.com/en/java/javase/25/docs/api/index.html>.

[3] S. Huber. „Design Patterns in der Softwareentwicklung: Die wichtigsten Muster erklärt.“ de, besucht am 1. Juli 2026. Adresse: <https://sascha-huber.com/allgemein/design-patterns-erklart/>.

[4] H. Balzert. „Objektmodellierung - Analysemuster und Beispielanwendungen.“ de, besucht am 1. Juli 2026. Adresse: https://www.entrap.de/study/5.Semester_WS0405/se/material/SE/Microsoft%20PowerPoint%20-%20le05.pdf.